Наконец, рассмотрим возможность построения **представлений** – виртуальных таблиц, которые представимы как результат выполнения некоторого запроса. Представления в дальнейшем можно использовать в других запросах как таблицы, понимая при этом, что каждый раз при обращении к представлению производится выполнение запроса, по которому представление было создано.

Например, создадим представление, в котором находится информации о том, какие зачеты и экзамены должен сдать в первом семестре каждый студент.

```
CREATE VIEW Student_Session
AS
SELECT FIOStudent, TitleSubject FROM Students INNER JOIN Sessions
       ON Students.NumGroup=Sessions.NumGroup INNER JOIN Subjects
       ON Subjects.idSubject=Sessions.idSubject
       WHERE NumSemestr=1;
```

Далее обратимся к этому представлению как к таблице. Например, найти те зачеты-экзамены, которые должен сдать студент Иванов:

```
SELECT TitleSubject FROM Student_Session
                     WHERE FIOStudent LIKE 'Иванов%';
```

1.5. ХРАНИМЫЕ ПРОЦЕДУРЫ, ФУНКЦИИ И ТРИГГЕРЫ

Хранимые процедуры, функции и триггеры вводятся в базу данных для обеспечения бизнес-логики приложения на уровне серверной его компоненты. Обычно *хранимые процедуры и функции* представляют собой утилиты, которые определенным образом обрабатывают данные или реализуют достаточно сложный алгоритм вычисления некоторых показателей.

Триггеры – это частный случай хранимой процедуры, который выполняется автоматически при выполнении команд обновления данных (INSERT, DELETE, UPDATE). Триггеры привязываются к конкретным таблицам базы данных. Для каждой команды должны быть свои триггеры.

В дереве элементов базы данных в любом СУБД имеются группы для определения этих программных элементов:

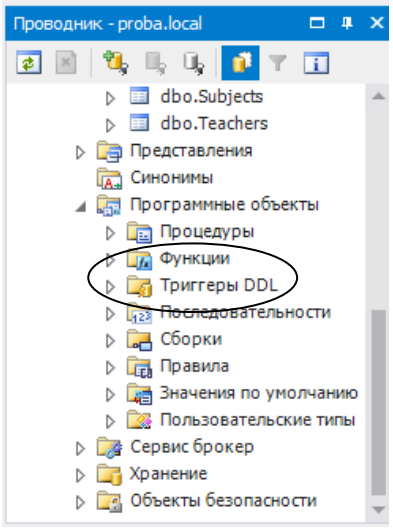

Рис. 32. Дерево элементов в MS SQL Server.

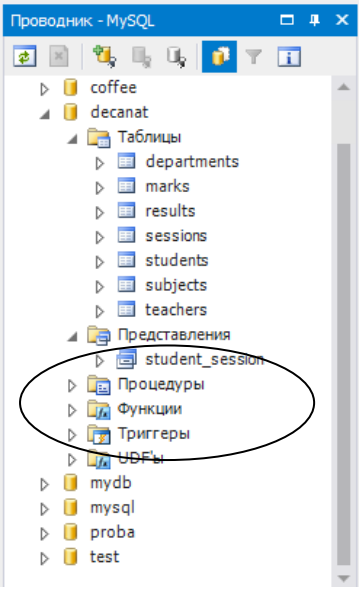

Рис. 33. Дерево элементов в MySQL.

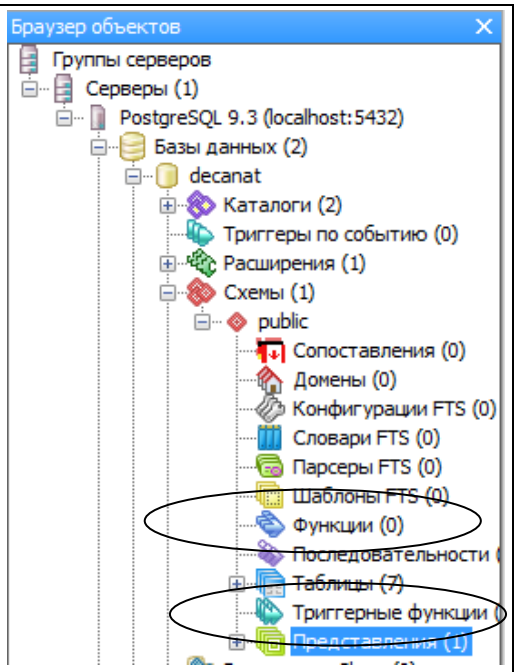

Рис. 34. Дерево элементов в PostgreSQL.

Для создания процедуры, функции или триггера требуется воспользоваться контекстным меню соответствующего элемента дерева. Для создания и редактирования существует специальное диалоговое окно, в котором, в частности, можно задать программный код процедуры, функции или триггера. Программный код формируется посредством перемешивания команд управления и SQL-команд.

Теперь приведем несколько примеров создания хранимых процедур и функций. Здесь мы уже заметим существенные отличия в синтаксисе используемых команд для различных СУБД. Поэтому для каждого из СУБД текст процедур, функций, триггеров и способы вызова укажем отдельно.

Пример 1. Напишем хранимую процедуру, которая получает в качестве входного параметра количество баллов и на основании шкалы оценок вычисляет полученную оценку. Результат возвращается через выходной параметр.

MS SQL Server. В SQL Server любая переменная именуется, начиная с символа ‘@’. Остальной код комментировать не требуется.

```
CREATE PROCEDURE dbo.GetMark1 (@ball int, @mark INT OUT)
AS
BEGIN
    IF @ball BETWEEN 55 AND 70
```

```
        SET @mark=3;
    ELSE IF @ball BETWEEN 71 AND 85
        SET @mark=4;
    ELSE IF @ball BETWEEN 86 AND 100
        SET @mark=5;
    ELSE SET @mark=2;
END
GO
```

Для вызова процедуры требуется создать переменную для применения ее в качестве выходного параметра, после чего воспользоваться командой EXEC. Распечатать результат в выходном потоке можно с помощью оператора PRINT.

```
-- пример вызова процедуры GetMark1
DECLARE @mark INT;
EXEC GetMark1 78, @mark OUT;
PRINT  '78 баллов соответствует оценке' + STR(@mark);
```

```
78 баллов соответствует оценке         4
Выполнение завершено успешно [0,112с]
```

Рис. 35. Результат выполнения хранимой процедуры в MS SQL Server.

MySQL. Принципиальных отличий в программном коде нет. Стоит отметить, что символ ‘@’ здесь используется только для глобальных переменных, поэтому имена параметров этот символ не имеют.

```
CREATE DEFINER = 'root'@'localhost'
PROCEDURE decanat.GetMark1(in ball INT, out mark INT)
BEGIN
    IF ball BETWEEN 55 AND 70 THEN
        SET mark=3;
    ELSEIF ball BETWEEN 71 AND 85 THEN
        SET mark=4;
    ELSEIF ball BETWEEN 86 AND 100 THEN
        SET mark=5;
    ELSE SET mark=2;
    END IF;
END
```

Вызов процедуры осуществляется следующим образом (достаточно отличным от MS SQL Server). В MySQL не предусмотрено окно сообщений, поэтому вывод осуществляется посредством выборки значения переменной:

```
call GetMark1(89,@m);
select ""+@m as "Оценка";
```

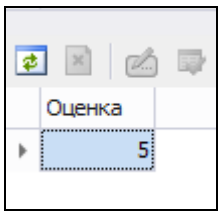

Рис. 36. Результат выполнения хранимой процедуры в MySQL.

PostgreSQL. Это СУБД не позволяет создавать процедуры. Здесь используются только функции. Еще одна особенность состоит в том, что функцию можно написать на разных языках. Наиболее распространены sql и plpgsql. Основное отличие языков состоит в том, что в sql доступны только операторы sql, а plpgsql имеет также операторы управления. Интересно, что именовать параметры вовсе не обязательно. К параметрам можно обращаться по номерам, предваренным символом “$”. Итак, создадим скрипт, в котором запишем следующую функцию:

```
CREATE FUNCTION GetMark1 (integer) RETURNS integer AS $$
DECLARE res INTEGER;
BEGIN
    IF $1 BETWEEN 55 AND 70 THEN
        SELECT 3 INTO res;
    ELSE IF $1 BETWEEN 71 AND 85 THEN
        SELECT 4 INTO res;
    ELSE IF $1 BETWEEN 86 AND 100 THEN
        SELECT 5 INTO res;
    ELSE SELECT 2 INTO res;
    END IF; END IF; END IF;
    RETURN res;
END;
$$  LANGUAGE plpgsql;
```

Отметим применение символов “$$” в начале и конце функции. Они позволяют игнорировать символы-разделители внутри этих своеобразных скобок. Функция декларирует тип возвращаемого значения с помощью ключевого слова RETURNS. В теле функции создается переменная для хранения результата, которой присваивается значение в зависимости от ветки условных операторов, по которой пойдет управление. К параметру производится обращение посредством “$1”. Отметим еще, что в конце следует указать используемый язык написании функции.

Вызов функции:

```
select GetMark1(68);
```

Пример 2. Чтобы при смене правил вычисления оценок не нужно было бы менять процедуру, мы создали справочную таблицу для хранения всех оценок и их диапазонов Marks. Пришло время ею воспользоваться. Второй вариант функции получения оценки по набранным баллам будет обращаться к этой таблице за информацией.

Оформим этот вариант в виде функции с одним параметром, хранящим набранные баллы, и возвращающую найденную оценку или 2 в случае, когда набранным баллам ничего в таблице не соответствует. В данной функции демонстрируется использование переменных, запросов и условного оператора. Приведем два варианта функции. С помощью запроса на количество таких записей алгоритм первого варианта предусматривает определение, есть ли в таблице соответствующая баллам оценка Второй вариант пользуется специальной функцией EXISTS, которая, принимая в качестве аргумента запрос, возвращает логическое значение, определяющее, есть ли в результате запроса записи.

MS SQL Server. В предыдущем примере мы уже видели одно из отличий языка для MS SQL Server, связанное с присвоением значений переменным. В MS SQL Server для этого предназначен оператор SET. В других рассматриваемых нами СУБД в этой роли выступает оператор SELECT.

Итак, первый вариант функции. В нем определяются две переменные для хранения количества записей и оценки. Значением оценки по умолчанию является оценка 2. После определяется количество записей в таблице Marks, соответствующее набранным баллам и, если это количество больше 0, найденная оценка присваивается переменной @mark, которая в конце возвращается как результат функции. Отметим, что оператор RETURN должен быть последним в функции.

```
CREATE FUNCTION dbo.GetMark2(@ball int)
RETURNS INT
  AS
  BEGIN
    DECLARE @kolvo INT, @mark INT;
    SET @mark=2;
    SET @kolvo=(SELECT COUNT(*) FROM Marks WHERE
```

```
                                @ball between LowBalls and HighBalls);
    IF @kolvo>0
      SET @mark=(SELECT idMark FROM Marks WHERE
                                @ball between LowBalls and HighBalls);
    RETURN @mark;
  END
GO
```

Во втором варианте функции в условном операторе вместо переменной @kolvo используется вызов функции EXISTS.

```
CREATE FUNCTION dbo.GetMark3(@ball int)
RETURNS INT
  AS
  BEGIN
    DECLARE @mark INT;
    SET @mark=2;
    IF EXISTS(SELECT * FROM Marks WHERE @ball
                        between LowBalls and HighBalls)
      SET @mark=(SELECT idMark FROM Marks WHERE
                   @ball between LowBalls and HighBalls);
    RETURN @mark;
  END
GO
```

Вызов функции оформляется следующим образом:

```
DECLARE @mark INT;
SET @mark=dbo.GetMark3(93);
PRINT  '93 балла соответствует оценке' + STR(@mark);
```

MySQL. Аналогичные функции для сервера MySQL определяются следующим образом:

```
CREATE FUNCTION decanat.GetMark2(ball int)
  RETURNS int(11)
BEGIN
    DECLARE kolvo, mark INT;
    SELECT 2 INTO mark;
    SELECT COUNT(*) INTO kolvo FROM Marks WHERE
                    ball between LowBalls and HighBalls;
    IF kolvo>0 THEN
      SELECT idMark INTO mark FROM Marks WHERE
                    ball between LowBalls and HighBalls;
    END IF;
    RETURN mark;
END


CREATE FUNCTION decanat.GetMark3(ball int)
  RETURNS int(11)
BEGIN
    DECLARE mark INT;
    SELECT 2 INTO mark;
    IF EXISTS (SELECT * FROM Marks WHERE
                        ball between LowBalls and HighBalls) THEN
```

```
      SELECT idMark INTO mark FROM Marks WHERE
                        ball between LowBalls and HighBalls;
    END IF;
    RETURN mark;
END
```

Вызов функции можно осуществлять непосредственно в выражении, например:

```
  SELECT ""+GetMark2(89) as "Оценка";
```

PostgreSQL. Как уже было сказано, в PostgreSQL хранимых процедур нет, в этом СУБД используются только функции. Программный код практически не будет отличаться от кода для MySQL за исключением определения переменных за скобками функции, обращения к параметру и обращения к полям и таблице базы данных:

```
CREATE FUNCTION GetMark2 (integer) RETURNS integer AS $$
DECLARE kolvo INTEGER;
DECLARE mark INTEGER;
BEGIN
    SELECT 2 INTO mark;
    SELECT COUNT(*) INTO kolvo FROM "Marks"
                   WHERE $1 between "LowBalls" and "HighBalls";
    IF kolvo>0 THEN
      SELECT "idMark" INTO mark FROM "Marks"
                    WHERE $1 between "LowBalls" and "HighBalls";
    END IF;
    RETURN mark;
END;
$$  LANGUAGE plpgsql;


CREATE FUNCTION GetMark3 (integer) RETURNS integer AS $$
DECLARE mark INTEGER;
BEGIN
    SELECT 2 INTO mark;
    IF  EXISTS (SELECT * FROM "Marks"
                WHERE $1 between "LowBalls" and "HighBalls") THEN
      SELECT "idMark" INTO mark FROM "Marks"
                              WHERE $1 between "LowBalls" and "High-
Balls";
    END IF;
    RETURN mark;
END;
$$  LANGUAGE plpgsql;
```

Теперь приведем примеры **создания триггеров.**

Пример 1. Создадим триггер для вставки в таблицу результатов сессии, в котором проверяются ограничения целостности (студент с заданном кодом существует, предмет с заданным кодом существует, дисциплину нужно сдавать

именно в этом семестре). Если произойдет нарушение этих ограничений, то требуется откатить транзакцию, т.е. не выполнять вставку записи. Если же все данные будут корректными, проведем заполнение значений полей даты сдачи зачета/экзамена как текущей и вычислим оценку по указанным баллам.

Для проверки корректности данных для вставки создадим вспомогательную хранимую функцию, чтобы код триггера был не очень сложным. (Для некоторых версий СУБД требуется, чтобы в триггере было упоминание только текущей записи, обращение к другим таблицам и записям недоступно).

MS SQL Server:

```
CREATE FUNCTION dbo.IsCorrect(@idStud INT, @idSubj INT,
                              @Sem INT, @idTeach INT) RETURNS INT
  AS
  BEGIN
    IF EXISTS (SELECT * FROM Students INNER JOIN Sessions
                       ON Students.NumGroup=Sessions.NumGroup
                       INNER JOIN Subjects ON
                       Sessions.idSubject=Subjects.idSubject
                       INNER JOIN Teachers ON
                       Sessions.idTeacher=Teachers.idTeacher
                       WHERE Students.idStudent=@idStud AND
                       Subjects.idSubject=@idSub
                       AND Teachers.idTeacher=@idTeach and NumSemestr=@Sem)
        RETURN 1;
    RETURN 0;
  END
GO
```

MySQL:

```
CREATE FUNCTION IsCorrect (idStud INT, idSubj INT, Sem INT, idTeach INT)
       RETURNS INT(11)
  BEGIN
      RETURN EXISTS (SELECT * FROM Students INNER JOIN Sessions
                     ON Students.NumGroup=Sessions.NumGroup
                     INNER JOIN Subjects ON
                     Sessions.idSubject=Subjects.idSubject
                     INNER JOIN Teachers ON
                     Sessions.idTeacher=Teachers.idTeacher
                     WHERE Students.idStudent=idStud AND
                     Subjects.idSubject=idSubj
                     AND Teachers.idTeacher=idTeach and NumSemestr=Sem);
  END
```

PostgreSQL:

```
CREATE FUNCTION IsCorrect(integer, integer, integer, integer)
                                                    RETURNS BOOLEAN AS $$
BEGIN
      RETURN EXISTS (SELECT * from "Students" INNER JOIN "Sessions"
                     ON "Students"."NumGroup"="Sessions"."NumGroup"
                     INNER JOIN "Subjects" ON
                     "Sessions"."idSubject"="Subjects"."idSubject"
```

```
                    INNER JOIN "Teachers" ON
                    "Sessions"."idTeacher"="Teachers"."idTeacher"
                    WHERE "Students"."idStudent"=$1 AND
                    "Subjects"."idSubject"=$2
                    AND "Teachers"."idTeacher"=$3 AND "NumSemestr"=$4);
END;
$$ LANGUAGE plpgsql;
```

Триггер на вставку записи в таблицу Results будет вызывать функцию проверки корректности, передавая в функцию поля из новой записи. Если запись будет корректной, будут скорректированы поля оценки и даты сдачи зачета/экзамена. В противном случае должен быть произведен откат транзакции.

MS SQL Server:

При вставке записи сначала запись попадает в виртуальную таблицу inserted (при удалении будет использоваться таблица deleted, при изменении записи используются обе таблицы – в inserted хранятся новые значения записи, в deleted – прежние значения полей записи). Поэтому сначала получаем данные новой записи из таблицы inserted, после чего проверяем их на корректность. В случае корректных данных оставшиеся поля (дата и оценка) изменяются посредством команды UPDATE. Откат транзакции в случае некорректных данных производится с помощью команды ROLLBACK.

```
CREATE TRIGGER trigger1
  ON dbo.Results
  FOR INSERT
AS
BEGIN
  -- объявление необходимых переменных для хранения данных новой записи
  DECLARE @idStudent INT, @idSubject INT,
          @idTeacher INT, @NumSemestr INT, @Balls INT;

  -- чтение данных новой записи
  SET @idStudent =(SELECT idStudent FROM inserted);
  SET @idSubject =(SELECT idSubject FROM inserted);
  SET @idTeacher =(SELECT idTeacher FROM inserted);
  SET @NumSemestr =(SELECT NumSemestr FROM inserted);
  SET @Balls =(SELECT Balls FROM inserted);

  -- проверка на корректность данных
  IF dbo.IsCorrect(@idStudent, @idSubject, @NumSemestr, @idTeacher)=0
    BEGIN
      -- данные некорректны. Выводим сообщение об ошибке
      -- и производим откат транзакции
      PRINT 'Ошибка данных: данные некорректны';
      ROLLBACK;
    END
  ELSE
     -- изменение полей даты и вычисление оценки.
```

```
        -- В условии указывается первичный ключ
        UPDATE dbo.Results SET mark=dbo.GetMark3(@Balls), DateExam=GETDATE()
                        WHERE idStudent=@idStudent AND idSubject=@idSubject
                        AND idTeacher=@idTeacher AND NumSemestr=@NumSemestr;
    END
    GO
```

MySQL:

Для MySQL данный триггер запишется проще, так как здесь проще получить данные новой записи. Новая запись хранится в виде объекта New (запись при удалении хранится в виде объекта Old). Однако имеется проблема, связанная с отсутствием команды отката триггера. В этом случае рекомендуется выполнить какую-нибудь ошибочную команду, например, вставить запись с уже существующим ключом. Ошибка в этой команде приведет к отмене действий всей транзакции (команды и триггера).

```
CREATE TRIGGER decanat.trigger1
        BEFORE INSERT
        ON decanat.results
        FOR EACH ROW
BEGIN
      IF IsCorrect(New.idStudent, New.idSubject, New.NumSemestr, New.idTeacher)
      THEN
        SET New.Mark=GetMark3(New.Balls);
        SET New.DateExam=Now();
      ELSE
         insert into Departments values (1,"","");
      END IF;
END
```

PostgreSQL:

В PostgreSQL триггер как таковой связан со специальной триггерной функцией, в которой и осуществляется вся обработка данных. Триггерная функция возвращает объект-запись (NEW или OLD), с которой производится работа. При написании триггера мы указываем только для какой операции, для какой таблицы и каков тип триггера, после чего вызываем триггерную функцию. Откат производится генерацией исключительной ситуации с указанием сообщения об ошибке. В остальном код похож на тот, что писался для MySQL:

```
-- Создание триггерной функции на вставку результата сдачи экзамена
CREATE FUNCTION trigger_results_insert() RETURNS trigger AS $$
BEGIN
  IF IsCorrect(NEW."idStudent", NEW."idSubject",
                                NEW."idTeacher", NEW."NumSemestr")
      THEN
        SELECT GetMark3(NEW."Balls") INTO NEW."Mark";
```

```
        SELECT Now() INTO New."DateExam";
      ELSE
         -- генерация исключительной ситуации
         RAISE EXCEPTION 'Ошибка корректности данных';
      END IF;
   RETURN NEW;
END;
$$ LANGUAGE  plpgsql;

-- Создание триггера на вставку нового результата экзамена
CREATE TRIGGER tr_results_insert
BEFORE INSERT ON "Results" FOR EACH ROW
EXECUTE PROCEDURE trigger_results_insert();
```

Для проверки работы триггера (например, для MySQL) проведем следующие операции вставки:

```
INSERT INTO Results (idStudent, idSubject,idTeacher, NumSemestr, Balls)
VALUES (1,1,1,1,78);
INSERT INTO Results (idStudent, idSubject,idTeacher, NumSemestr, Balls)
VALUES (2,1,1,1,98);
INSERT INTO Results (idStudent, idSubject,idTeacher, NumSemestr, Balls)
VALUES (6,1,1,1,68);
```

Согласно данным, которые мы вносили в таблицу, последняя запись не должна быть добавлена.

Пример 2. Приведем еще один пример триггера на вставку новой записи в таблицу результатов. Этот триггер должен срабатывать после вставки и быть связан с подсчетом рейтинга студентов. Триггеры «после» часто используются для проведения специальной обработки данных на основании выполненной операции и могут быть связаны с другими таблицами.

Для этого введем в базу данных новую таблицу, например, с помощью следующей SQL-команды:

```
CREATE TABLE Reyting
  ( idStudent INT PRIMARY KEY,
    summ_balls INT,
    CONSTRAINT fk_reyting
               FOREIGN KEY (idStudent) REFERENCES Students (idStudent)
  )
```

При вставке нового результата рейтинг студента должен меняться. Таким образом, нужно проанализировать, есть ли запись о студенте – в случае положительного ответа произвести суммирование баллов, иначе добавить новую запись в таблицу рейтинга.

MS SQL Server:

```
CREATE TRIGGER trigger2
ON dbo.Results
AFTER INSERT
AS
  BEGIN
    DECLARE @idStudent INT, @Balls INT;
    SET @idStudent = (SELECT idStudent FROM inserted);
    SET @Balls =(SELECT Balls FROM inserted);

    IF EXISTS(SELECT * FROM Reyting WHERE idStudent=@idStudent)
      UPDATE Reyting SET summ_balls=summ_balls+@Balls
                                    WHERE idStudent=@idStudent;
    ELSE
      INSERT INTO Reyting (idStudent, summ_balls)
                                    VALUES (@idStudent, @Balls);
  END
GO
```

MySQL:

```
CREATE TRIGGER decanat.trigger2
  AFTER INSERT
  ON decanat.results
  FOR EACH ROW
BEGIN
  IF EXISTS(SELECT * FROM Reyting WHERE idStudent=new.idStudent) THEN
      UPDATE Reyting SET summ_balls=summ_balls+new.Balls
                     WHERE idStudent=new.idStudent;
  ELSE
      INSERT INTO Reyting (idStudent, summ_balls)
                          VALUES (New.idStudent, New.balls);
  END IF;
END
```

PostgreSQL:

```
CREATE FUNCTION trigger_results_insert_after() RETURNS trigger AS $$
BEGIN
 IF EXISTS(SELECT * FROM "Reyting" WHERE "idStudent"=NEW."idStudent") THEN
      UPDATE "Reyting" SET "summ_balls"="summ_balls"+NEW."Balls"
                       WHERE "idStudent"=NEW."idStudent";
  ELSE
      INSERT INTO "Reyting" ("idStudent", "summ_balls")
                           VALUES (NEW."idStudent", NEW."Balls");
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE  plpgsql;

CREATE TRIGGER tr_results_insert_after
AFTER INSERT ON "Results" FOR EACH ROW
EXECUTE PROCEDURE trigger_results_insert_after();
```

Поэкспериментируйте сами со вставками записей, чтобы изменялся рейтинг студентов.

Разберем еще один пример хранимой функции для демонстрации использования **курсоров** – временных таблиц, представляющих собой результаты выполнения запроса и обрабатываемые построчно – от первой записи до последней. Для этого создадим еще одну версию функции перевода баллов в оценку – каждая строка таблицы Marks в ней будет обрабатываться построчно до получения строки с нужной оценкой или отсутствием соответствующей оценки.

MS SQL Server:

```
CREATE FUNCTION dbo.GetMark4(@balls INT)
RETURNS INT
  BEGIN
     DECLARE @res INT, @mark INT, @lowB INT, @highB INT;
     SET @res=2;

     -- декларация курсора, связанного с запросом
     DECLARE mark_cursor CURSOR FOR SELECT * FROM Marks;

     -- открытие курсора
     OPEN mark_cursor;
     -- считывание первой строки курсора в переменные @mark, @lowB, @highB
     FETCH NEXT FROM mark_cursor INTO @mark, @lowB, @highB;

     -- цикл продолжается, пока считывание возможно,
     -- на это укажет глобальная переменная
     WHILE @@FETCH_STATUS = 0
       BEGIN
         -- определяем, соответствуют ли баллы текущей оценке
         IF @balls BETWEEN @lowB AND @highB
         BEGIN
            SET @res=@mark;
            BREAK;
         END
         -- переход к следующей строке курсора
         FETCH NEXT FROM mark_cursor INTO @mark, @lowB, @highB;
       END

     -- закрытие курсора
     CLOSE mark_cursor;
     -- разрушение курсора
     DEALLOCATE mark_cursor;
     RETURN @res;
  END
GO
```

MySQL: для обработки завершения записей курсора здесь требуется создать специальный обработчик CONTINUE HANDLER FOR NOT FOUND. В остальном работа с курсором аналогична.

```
CREATE DEFINER = 'root'@'localhost'
FUNCTION decanat.GetMark4(balls INT)
  RETURNS int(11)
BEGIN
    -- переменные для хранения полей кортежа из таблицы Marks
    DECLARE mark, lowB, highB, res INT;
```

```
    -- переменная для определения, завершен ли просмотр курсора
    DECLARE is_end INT DEFAULT 0;
    -- определение курсора для таблицы Marks
    DECLARE mark_cursor CURSOR FOR SELECT * FROM Marks;

    -- объявление обработчика ошибки  завершея записей курсора
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET is_end=1;

    SET res=2;
     -- открытие курсора
    OPEN mark_cursor;
    -- считывание первой записи курсора
    FETCH mark_cursor INTO mark, lowB, highB;

    -- организация цикла просмотра строк из курсора
    WHILE is_end=0 DO

       -- проверка диапазона баллов для текущей оценки
       IF balls BETWEEN lowB AND highB THEN
           SET res=mark;
           -- организация выхода из цикла
           SET is_end=1;
        END IF;
         -- считывание очередной записи курсора
         FETCH mark_cursor INTO mark, lowB, highB;
    END WHILE;
    CLOSE mark_cursor;
    RETURN res;
  END
```

PostgreSQL: как и в предыдущем случае отличия будут в организации цикла и выхода из него.

```
CREATE FUNCTION GetMark4 (integer) RETURNS integer AS $$
DECLARE res integer;
DECLARE mark integer;
DECLARE lowB integer;
DECLARE highB integer;
DECLARE mark_cursor CURSOR FOR SELECT * FROM "Marks";
BEGIN
    res:=2;
    OPEN mark_cursor;--открываем курсор
    LOOP --начинаем цикл по курсору
      --извлекаем данные из строки и записываем их в переменные
      FETCH mark_cursor INTO mark, lowB, highB;
      --если такого периода и не возникнет, то мы выходим
      IF NOT FOUND THEN EXIT;END IF;
      IF $1 BETWEEN lowB AND highB THEN
           res:=mark;
      END IF;
    END LOOP;--заканчиваем цикл по курсору
  CLOSE mark_cursor; --закрываем курсор
  return res;
 END;
$$  LANGUAGE plpgsql;
```